🍔 EXAMEN FINAL - TIENDA DE COMIDAS

Desarrollo de Aplicaciones Web con REST API

Marzo 2026

⏱️ **Duración del Examen:** 4 horas
📊 **Evaluación:** Funcionalidad 40%, Código 30%, Integración BD 20%, UX 10%
🎯 **Objetivo:** Crear un sistema completo de e-commerce con Frontend + Backend + Base de Datos

1. INTRODUCCIÓN Y OBJETIVOS

En este examen deben demostrar sus habilidades en desarrollo web full-stack implementando un sistema de tienda de comidas con carrito de compras, integración de APIs REST y persistencia en base de datos MySQL.

Objetivos Específicos:

- Correr correctamente el servidor backend Node.js
- Implementar CRUD completo en el frontend (Dashboard)
- Crear un carrito de compras funcional con localStorage
- Realizar peticiones fetch() a la API REST
- Guardar pedidos en la base de datos
- Validar formularios y manejar errores

2. CONFIGURACIÓN Y EJECUCIÓN DEL BACKEND

2.1 Requisitos Previos

- Node.js v14+ instalado
- MySQL Server corriendo
- npm disponible en la terminal

2.2 Pasos para Ejecutar el Backend

Paso 1: Abrir la terminal en la carpeta del backend

```
cd C:\ruta\a\BACKEND_TIENDA_NODE_MYSQL
```

Paso 2: Instalar dependencias (solo la primera vez)

```
npm install
```

Paso 3: Configurar la base de datos

El archivo `.env` debe estar configurado así:

```
DB_HOST=localhost DB_USER=root DB_PASSWORD= DB_NAME=tienda_db PORT=3000
```

Si la base de datos no existe, ejecutar el script de inicialización:

```
node scripts/init-db.js
```

Paso 4: Iniciar el servidor

```
npm start
```

O con nodemon (modo desarrollo):

```
npm run dev
```

✅ **Éxito:** Debe aparecer: `Servidor corriendo en puerto 3000`

2.3 Endpoints Disponibles

El backend proporciona estos endpoints para el examen:

Módulo	Método	Endpoint	Descripción
Productos	GET	`/api/productos`	Obtener todos los productos
	POST	`/api/productos`	Crear nuevo producto
	PUT	`/api/productos/:id`	Actualizar producto
	DELETE	`/api/productos/:id`	Eliminar producto
Clientes	GET	`/api/clientes`	Obtener todos los clientes
	POST	`/api/clientes`	Crear nuevo cliente
	PUT	`/api/clientes/:id`	Actualizar cliente
	DELETE	`/api/clientes/:id`	Eliminar cliente
Pedidos	GET	`/api/pedidos`	Obtener todos los pedidos
	POST	`/api/pedidos`	Crear nuevo pedido
	PUT	`/api/pedidos/:id`	Actualizar pedido
	PATCH	`/api/pedidos/:id/estado`	Cambiar estado del pedido
	DELETE	`/api/pedidos/:id`	Eliminar pedido

2.4 Solución de Problemas Comunes

❌ **Error: "Cannot find module"**
Solución: Ejecutar `npm install`

❌ **Error: "Connection refused"**
Solución: Verificar que MySQL esté corriendo

❌ **Error: "Database does not exist"**
Solución: Ejecutar `node scripts/init-db.js`

3. INSTRUCCIONES - MÓDULO DASHBOARD (Frontend-apicrud)

📊 **Gestiones Administrativas**

Los estudiantes deben implementar el CRUD (Create, Read, Update, Delete) para 4 módulos

3.1 Módulo de Productos

Archivo: `listado-pro.html` y `crear-pro.html`

✅ **Tareas a Implementar:**

- Cargar lista de productos desde `GET /api/productos`
- Mostrar en tabla: ID, Nombre, Precio, Descripción, Acciones
- Botones Editar y Eliminar funcionales
- Crear nuevo producto con formulario
- Validación de campos requeridos
- Actualizar producto con PUT
- Mostrar mensajes de éxito/error

Estructura de Producto (JSON):

```
{ "id_producto": 1, "nombre": "Hamburguesa Clásica", "precio": 10.50,
"descripcion": "Hamburguesa de carne con lechuga y tomate", "imagen":
"hamburguesa.jpg", "disponible": 1 }
```

3.2 Módulo de Clientes

Archivo: `listado-clientes.html` y `crear-cliente.html`

✅ **Tareas a Implementar:**

- Cargar lista de clientes desde `GET /api/clientes`
- Mostrar en tabla: ID, Nombre, Apellido, Email, Celular, Acciones
- Crear nuevo cliente
- Actualizar datos de cliente
- Eliminar cliente con confirmación
- Validar email válido
- Buscar/filtrar clientes

Campos del Cliente:

```
{ "id_cliente": 1, "nombre": "Juan", "apellido": "Pérez", "email":
"juan@example.com", "celular": "3001234567", "direccion": "Calle 10 #20-30",
"descripcion": "Cliente frecuente" }
```

3.3 Módulo de Pedidos

Archivo: `listado-pedidos.html` y `crear-pedido.html`

✅ **Tareas a Implementar:**

- Cargar lista de pedidos desde `GET /api/pedidos`
- Mostrar en tabla: ID, Cliente, Email, Fecha, Total, Estado, Acciones
- Cambiar estado del pedido (PATCH /api/pedidos/:id/estado)
- Estados válidos: pendiente, procesando, completado, cancelado
- Crear nuevo pedido con múltiples productos
- Seleccionar cliente desde dropdown
- Agregar productos dinámicamente
- Calcular total automáticamente
- Aplicar descuentos y costos de domicilio

Estructura de Pedido:

```
{ "id_pedido": 1, "id_cliente": 1, "metodo_pago": "contra-entrega", "estado":
"pendiente", "total": 25.50, "fecha": "2026-03-01", "productos": [ {
```

```
"id_producto": 1, "cantidad": 2, "subtotal": 21.00 } ] }
```

3.4 Módulo de Usuarios

Archivo: `listado-usuarios.html` y `crear-usuario.html`

✅ **Tareas a Implementar:**

Cargar lista de usuarios desde `GET /api/usuarios`

Mostrar en tabla: ID, Usuario, Rol, Fecha Creación, Acciones

Roles: administrador, vendedor, cajero

Crear usuario con rol

Validar contraseña (mínimo 6 caracteres)

Confirmar contraseña coincide

Actualizar usuario

Eliminar usuario

Estructura de Usuario:

```
{ "id_usuario": 1, "usuario": "admin", "contrasena": "hash_encriptado", "rol":
"administrador", "creado_en": "2026-01-15" }
```

3.5 Requisitos Técnicos del Dashboard

- ✅ Todas las peticiones deben ir a `http://localhost:3000/api/`
- ✅ Usar `fetch()` para todas las peticiones
- ✅ Manejar errores de conexión con try/catch
- ✅ Validar respuestas HTTP (200, 404, 500)
- ✅ Mostrar mensajes de error/éxito al usuario
- ✅ Código limpio y bien documentado
- ✅ Funciones reutilizables

4. INSTRUCCIONES - PÁGINA DE COMPRAS (platilla-carrito)

🛒 **Sistema de Carrito de Compras**

Los estudiantes deben crear una tienda funcional con carrito y checkout

4.1 Página Principal - index.html

Objetivo: Mostrar catálogo de productos y agregar al carrito

✅ **Tareas a Implementar:**

Cada producto tiene atributos: `data-id, data-price, data-name, data-image`

Click en ".btn-product" debe agregar al carrito

Guardar carrito en localStorage

Actualizar contador de carrito (.contar-pro)

Mostrar notificación al agregar producto

Navegar a cart.html desde botón

Ejemplo de JavaScript:

```
// Obtener todos los botones de agregar document.querySelectorAll('.btn-product').forEach(btn => { btn.addEventListener('click', (e) => { const card = e.target.closest('.card.producto'); const id = card.dataset.id; const nombre = card.dataset.name; const precio = card.dataset.price; // TODO: Agregar al carrito }); });
```

4.2 Página de Carrito - cart.html

Objetivo: Visualizar, modificar y revisar el carrito

✅ **Tareas a Implementar:**

Cargar carrito desde localStorage

Mostrar tabla con: Imagen, Nombre, Precio, Cantidad, Subtotal

Botón eliminar producto

Cambiar cantidad con input numérico

Calcular subtotal automáticamente

Mostrar total del carrito

Botón "Continuar comprando" (volver a index)

Botón "Ir a pagar" (ir a checkout)

IDs a Usar:

```
- tabla-carrito: Tabla principal - carrito-items: tbody para las filas -
subtotal-resumen: Mostrar subtotal ($X.XX) - domicilio-resumen: Mostrar costo
envío - descuento-resumen: Mostrar descuentos - total-resumen: Mostrar TOTAL
```

4.3 Página de Checkout - checkout.html

Objetivo: Recolectar datos del cliente y procesar el pedido

✅ **Tareas a Implementar:**

Formulario con IDs específicos (nombres, apellidos, email, celular, etc)

Mostrar resumen del carrito dinámicamente

Seleccionar método de pago (contra-entrega, PSE, transferencia)

Validar que todos los campos estén llenos

Validar email válido

Calcular total con impuestos/domicilio

Botón "Place Order" hace POST a /api/pedidos

Mostrar error si falla la conexión

Redirigir a thankyou.html si es exitoso

Estructura POST a /api/pedidos:

```
{ "id_cliente": 1, "metodo_pago": "contra-entrega", "estado": "pendiente",
"total": 25.50, "notas": "Sin cebolla", "productos": [ {"id_producto": 1,
"cantidad": 2} ] }
```

4.4 Página de Confirmación - thankyou.html

Objetivo: Confirmar que el pedido fue recibido

✅ **Tareas a Implementar:**

Mostrar mensaje de éxito

Mostrar número de pedido generado

Mostrar tiempo estimado de entrega

Limpiar carrito de localStorage

Botón volver al inicio

4.5 Requisitos Técnicos del Carrito

- ✅ Usar **localStorage** para guardar carrito
- ✅ Estructura del carrito en localStorage:

```
const carrito = [ { id: 1, nombre: "Hamburguesa", precio: 10.50, cantidad: 2,
imagen: "./images/b1.png" } ];
```

- ✅ Usar `JSON.stringify()` para guardar
- ✅ Usar `JSON.parse()` para recuperar
- ✅ Calcular totales correctamente
- ✅ Validar formulario antes de enviar
- ✅ Manejar errores de red

5. CHECKLIST DE ENTREGA

Antes de Entregar, Verificar:

Backend ✅

Backend corre sin errores en puerto 3000

Base de datos creada y poblada

Todos los endpoints responden correctamente

CORS habilitado para peticiones frontend

Dashboard ✅

- Módulo Productos: CRUD completo
- Módulo Clientes: CRUD completo
- Módulo Pedidos: CRUD + cambio de estado
- Módulo Usuarios: CRUD
- Todas las páginas cargan datos correctamente
- Formularios validan datos
- Mensajes de error/éxito visibles
- Navegación entre módulos funciona

Tienda de Compras ✅

- Se pueden agregar productos al carrito
- Carrito persiste al cambiar de página
- Se puede modificar cantidad en carrito
- Se pueden eliminar productos del carrito
- Totales calculan correctamente
- Formulario checkout valida datos
- Pedido se guarda en base de datos
- Muestra página de confirmación

Código ✅

- Código bien indentado y legible
- Funciones documentadas con comentarios
- Variables con nombres descriptivos
- Sin errores en consola del navegador
- Manejo de errores implementado

6. PAUTAS DE EVALUACIÓN

Criterio	Porcentaje	Descripción
Funcionalidad	40%	¿Todas las operaciones CRUD funcionan? ¿El carrito y checkout son funcionales?
Integración API	20%	¿Las peticiones fetch() son correctas? ¿Se integra bien con el backend?
Persistencia BD	20%	¿Los datos se guardan en la base de datos? ¿Se guardan ordenadamente?
Código Limpio	15%	¿El código es legible? ¿Está bien organizado? ¿Tiene comentarios?
UX/Manejo de Errores	5%	¿Muestra mensajes claros? ¿Maneja errores de conexión?

7. RECURSOS Y AYUDA

Documentación Útil

- **Fetch API:** `https://developer.mozilla.org/es/docs/Web/API/Fetch_API`
- **localStorage:** `https://developer.mozilla.org/es/docs/Web/API/Storage`
- **REST API:** `https://restfulapi.net/`
- **Bootstrap 4:** `https://getbootstrap.com/docs/4.6/`

Comandos Útiles para Debug

```
// En la consola del navegador:
console.log(JSON.parse(localStorage.getItem('carrito'))); localStorage.clear();
// Limpiar todo fetch('http://localhost:3000/api/productos').then(r =>
r.json()).then(d => console.log(d));
```

Problemas Comunes y Soluciones

❌ **"Cannot GET /api/productos"**
Verificar que el endpoint existe y el backend está corriendo en puerto 3000

❌ **"CORS error"**
El backend tiene CORS habilitado, asegúrate de usar la URL correcta

❌ **"localStorage.getItem is null"**
Verificar que hayas guardado datos con localStorage.setItem() primero

❌ **"Esperaba JSON pero recibí HTML"**
La respuesta no fue exitosa. Verificar status HTTP en Network tab

8. CONSEJOS PARA EL ÉXITO

💡 **Tip 1:** Empieza por el módulo productos (ya tiene ejemplo de código)

💡 **Tip 2:** Prueba cada operación en la consola antes de implementarla

💡 **Tip 3:** Usa las Developer Tools (F12) para debuggear

💡 **Tip 4:** Valida los datos antes de enviar al servidor

💡 **Tip 5:** Implementa funcionalidad básica primero, luego mejora

💡 **Tip 6:** Documenta tu código con comentarios claros

💡 **Tip 7:** Guarda tu trabajo frecuentemente

💡 **Tip 8:** Si algo no funciona, usa console.log() para debuggear

---

© 2026 - Examen Final Desarrollo Web | Tienda de Comidas con REST API y MySQL

Para imprimir como PDF: Usar Ctrl+P o File → Print → Guardar como PDF